

RSLAB

A Pure-Rust Sparse Direct Solver: Algorithms, A-Priori Predictors, and a Learned Configuration Tuner

Milan Rother

July 1, 2026

Abstract

RSLAB is a pure-Rust sparse direct solver. It provides a supernodal complex-symmetric $\mathbf{LDL}^\top$ factorization with Bunch–Kaufman pivoting and an unsymmetric multifrontal LU, is generic over the scalar type (`f64`, `f32`, `Complex<f64>`, `Complex<f32>`), and links no external numeric library (no BLAS, LAPACK, or METIS): the SIMD dense kernels, the fill-reducing orderings, and the factorization are all Rust. The analyze phase computes, in addition to the symbolic factorization, a set of a-priori predictors that are exact functions of the matrix structure: a per-path peak-memory bound (separate for the left-looking and the multifrontal schedule), a geometric-flop work estimate, a runtime estimate from a hardware calibration, and a thread-count recommendation. The solver exposes its analyze, factor, and kernel parameters through one flat settings interface, and a learned configuration tuner selects a setting from the matrix structure, constrained by a deterministic guard stack that keeps its predicted memory below the untuned default and falls back to the default outside the training range. The numeric factorization is bit-identical regardless of the worker count, and the resource planner is a pure function of its inputs, which makes batched execution reproducible and its memory and runtime cost computable in advance. This report documents the algorithms, the predictors, the tunable stages, the determinism properties, and the solver-in-the-loop execution model, and reports the solver against `faer`, MKL PARDISO, and SuperLU over the SuiteSparse collection.

1 Scope

RSLAB factorizes and solves $Ax = b$ for sparse A that is either complex-symmetric (routed to $\mathbf{LDL}^\top$) or general unsymmetric (routed to LU). The primary target is the complex-symmetric systems of frequency-domain electromagnetic and FEM analysis, including curl–curl (Nédélec edge-element) Maxwell problems with lossy media or PML, which are complex-symmetric rather than Hermitian. The implementation is stable Rust with no C/Fortran dependency and no BLAS/LAPACK/METIS linkage, and is exposed to Python through a PyO3 layer.

2 Architecture

The solver follows the standard three-phase sparse-direct flow. *Analyze* orders the matrix, builds the elimination tree, detects and amalgamates supernodes, computes column counts, and produces the a-priori predictors (Sec. 6) and the structural feature vector (Sec. 6). *Factor* performs the numeric $\mathbf{LDL}^\top$ or LU on the ordered, equilibrated matrix. *Solve* runs the triangular substitutions. Analyze is separated from factor at the API level: a symbolic object (`LdlSymbolic` or `LuSymbolic`) is produced once and can factor many matrices that share its sparsity pattern, which is the basis of the solver-in-the-loop model (Sec. 10).

The code is a Cargo workspace (Table 1). The ordering crates share a common quotient-graph elimination engine; the numeric core holds the two factorization paths and the dense SIMD

kernels; the diagnostics, analysis, tuning, and `auto_tune` modules hold the predictors, the feature extraction, the resource planner, and the learned tuner respectively.

Crate / module	Responsibility
<code>rslab-ordering-core</code>	Shared quotient-graph elimination engine (workspace, sliding index allocator with garbage collection, supervariable detection)
<code>rslab-amd</code>	Approximate minimum degree
<code>rslab-amf</code>	Approximate minimum fill (HAMF4)
<code>rslab-metis</code>	Multilevel nested dissection (iterative driver, König separators)
<code>symbolic</code>	Elimination tree, supernode amalgamation, column counts, ordering dispatch
<code>numeric</code>	Supernodal $\mathbf{LDL}^\top$, multifrontal LU, panel-freeing, dense SIMD kernels, scoped pools
<code>diagnostics</code>	Per-path a-priori memory and flop estimates
<code>analysis</code>	Structural feature extraction, thread-count predictor
<code>tuning</code>	Hardware calibration, runtime estimate, budget-aware resource planner
<code>auto_tune</code>	Embedded performance model and guard stack (pure-Rust inference)
<code>python</code>	PyO3 bindings

Table 1: Workspace layout.

3 Fill-reducing orderings

The ordering determines fill and therefore both factor time and memory. RSLAB implements three ordering families behind an `OrderingMethod` enum and an adaptive dispatcher.

3.1 Approximate minimum degree

AMD [1] is a quotient-graph elimination that selects the pivot of least approximate external degree. The workspace holds the element and variable lists in a single sliding index array `iw`, with element pointers `pe`, list lengths `len/elen`, supervariable counts `nv`, running degrees `degree`, a generation-counter mark array `w`, and degree-indexed LIFO bucket lists `head/next/last`; when `iw` fills, an in-place garbage collection compacts the live lists. The elimination loop repeatedly selects the minimum-degree pivot, forms a new element by merging its element list with its variable tail (in place when the pivot has no prior elements, out of place otherwise), and updates the affected variables’ approximate degrees. During the update it performs aggressive element absorption, mass elimination of variables that become interchangeable with the pivot, and supervariable detection by hashing variables with identical adjacency and merging the indistinguishable ones. Variables of degree exceeding $\max(16, \lfloor \alpha \sqrt{n} \rfloor)$ with $\alpha = 10$ are deferred to the tail as dense rows. AMD is the default for very large, very sparse patterns, where the separator overhead of nested dissection does not pay off.

3.2 Approximate minimum fill

AMF [2, 3] replaces the degree score with an approximate *fill* score. Each supervariable i carries

$$\text{RMF}(i) = \frac{\deg(i) (\deg(i) - 1 + 2 \deg_{me}) - \text{WF}(i)}{nv(i) + 1}, \quad (1)$$

where $\text{WF}(i)$ is the accumulated working fill, updated per eliminated element by a surface term $\text{dext} (2 \deg - \text{dext} - 1)$ and combined across supervariables as $\text{wf}_4 + 2 nv_i \text{wf}_3$. The working fill is accumulated in 64-bit integers because the product is $\mathcal{O}(n^2)$ and overflows 32-bit for $n \gtrsim 46\,000$. Scores are placed in a bucket array of length $2n + 2$ with exact fine buckets $[0, n]$ and coarse buckets of stride $\max(n/8, 1)$ above; the pivot is the minimum exact score. Supervariable merges take the larger of the two working-fill values. AMF is the HAMF4 variant and runs aggressive

absorption unconditionally; it is the default for $n \leq 10\,000$, where it stays within $1.10\times$ of the reference HAMF4 fill on a 183 277-matrix oracle suite.

3.3 Nested dissection

The nested-dissection ordering [4, 5] is a multilevel recursive bisection driven by an explicit work stack of (subgraph, vertex map, offset) items rather than recursion, so deep dissection trees cannot overflow the call stack. Each subgraph is bisected by coarsening the graph, computing an initial bisection on the coarsest level, refining it with Fiduccia–Mattheyses [6], and uncoarsening with projection and refinement at each level. The vertex separator is extracted from the edge-cut bipartition by a minimum vertex cover, computed by König’s theorem: a maximum matching on the bipartite crossing-edge graph, then the alternating-path reachable set gives the cover. Two guards bound the recursion: subgraphs below `nd_to_amd_switch` = 200 vertices are ordered directly by AMD, and a split where one side holds $\geq 90\%$ of the vertices falls back to an AMD leaf rather than recursing on a graph without a good separator. Nested dissection is the default for $n > 10\,000$.

3.4 Dispatch

`OrderingMethod::Auto` routes by cheap structural predicates. A very large and sparse pattern ($n > 100\,000$, average degree < 5) goes to AMD. An arrow or bordered pattern goes to AMF, which defers the dense border to the trailing block where nested dissection would smear it across separators; the detector flags a pattern where at least 20% of the nonzeros are concentrated in fewer than 5% of the columns above a degree floor of $\max(64, 8\deg)$. Otherwise the size rule above applies. `OrderingMethod::AutoRace` instead runs the full symbolic factorization for each concrete candidate and keeps the one with the smallest factor nnz, at about $4\times$ the single-pass analysis cost.

4 Symbolic analysis

From the ordered pattern RSLAB builds the elimination tree and detects fundamental supernodes, which are maximal runs of columns whose row structure differs only by the eliminated column. Small supernodes are amalgamated to widen the dense fronts that feed the BLAS-3 kernels. Nodes with fewer than `nemin` = 16 columns [7] are merged with their parent; the merge order uses a column renumbering that re-postorders the tree so a desired parent–child merge becomes structurally adjacent, which is what makes amalgamation effective on bushy assembly trees. A shape predicate selects between the adjacency-only and the renumbering merge strategies, using the renumbering strategy unless the fraction of multi-child internal nodes is below 0.05 (a path-like tree, where renumbering would over-merge). Beyond that, *relaxed* amalgamation [8], applied for $n \geq 1024$, merges an adjacent child into its parent even when not fill-justified as long as the merged supernode stays within a width and extra-row budget, trading a little explicit-zero fill for wider, higher-rank Schur updates. Column counts are computed by a depth-first walk of the elimination tree, and drive the factor-size estimate.

5 Numeric factorization

5.1 Equilibration

Before factoring, A is symmetrically equilibrated as $\hat{A} = DAD$ with a real diagonal $D = \text{diag}(s)$, $s_i = 1/\sqrt{\max_j |A_{ij}|}$, computed in one pass over the lower triangle; an all-zero row is left unscaled. The real, symmetric, positive diagonal preserves complex symmetry and the pivot signs, tolerates zero diagonals (common in saddle-point systems), and improves the conditioning the bounded

pivoting sees. The solve unwinds $x = D(\hat{A}^{-1}(Db))$. The LU path applies a two-sided equilibration $\hat{A} = D_r A D_c$.

5.2 Supernodal left-looking $\mathbf{LDL}^\top$

The default path is a supernodal left-looking $\mathbf{LDL}^\top$. Each supernode holds a dense panel of size `nrow` $\times$ `ncol`, where `nrow` is the number of rows in its factored column structure. The panel is assembled from A , then updated by every previously factored descendant through the `cmod` operation, which pulls the descendant's columns, applies its block-diagonal D , and subtracts the resulting rank update; the update runs as a scalar triple loop below a flop threshold and as a SIMD GEMM above it. The fully-summed block is then factored in place by `cdiv`, a blocked Bunch–Kaufman partial factorization with panel width `panel_nb` = 64.

The Bunch–Kaufman kernel [9] factors the dense block as $P^\top A P = \mathbf{LDL}^\top$ with L unit lower triangular and D block-diagonal of 1×1 and 2×2 pivots. For complex-symmetric A the control flow is that of the real symmetric `sytf2` with magnitudes $|z|$ and no conjugation. The pivot threshold is $\alpha = (1 + \sqrt{17})/8 \approx 0.6404$; pivot selection is bounded to the panel's fully-summed rows so the factorization stays within the supernode. A 2×2 block is rejected as singular when $|d_{11}d_{22} - d_{21}^2| < \varepsilon \max(|d_{11}|, |d_{21}|, |d_{22}|)^2$ with $\varepsilon = 1 \times 10^{-14}$, which bounds the element growth injected into the trailing update. Each panel's trailing Schur update is deferred and routed through a single SIMD GEMM.

5.3 Panel freeing

The transient memory of the left-looking path is controlled by freeing each dense panel once its last consumer is done. Every supernode carries a reference count of the ancestors that will still update from it. When a descendant's `cmod` decrements that count to zero, the panel is compacted into its final CSC factor fragment and the dense buffer is released; the count is manipulated with acquire/release atomics so this is safe under the tree-parallel factorization. The resident data is therefore the compact factor plus only the live panels, not all panels at once. Sibling subtrees factor concurrently, and the per-node GEMMs parallelize above their flop thresholds.

5.4 Multifrontal path

The alternative multifrontal path [10, 11] assembles a dense frontal matrix per supernode from its eliminated columns and the contribution blocks (CBs) of its children, factors the fully-summed part, and passes the Schur complement to the parent as its CB. A work-stealing tree recursion factors children in parallel; each child's CB is freed the moment it is extend-added into the parent, so only the active contribution frontier is live. Where the child CB stack is large (at least 4×10^6 scalars) children are reordered by Liu's (peak – *cb*) rule [12] to shrink the transient peak while preserving leaf parallelism; the reordering is a scheduling hint and does not change the numbering, factor, or fill. The multifrontal front is denser and more BLAS-3-rich than a left-looking panel but carries a higher transient peak, which is why the two paths are separate and the choice is per matrix.

5.5 Unsymmetric LU

General matrices reuse the symmetric multifrontal machinery on the symmetrized pattern $A \cup A^\top$, with UMFPACK-style [13] threshold partial pivoting: within each front's fully-summed block a diagonal candidate is accepted when $|a_{jj}| \geq \tau |a_{\text{colmax}}|$ with $\tau = 0.1$, else the column maximum is swapped up. The panel width is `NB` = 32, narrower than the $\mathbf{LDL}^\top$ default because the serial within-panel factorization is the bottleneck while the SIMD GEMM is fast. L is stored CSC (unit lower), U CSR; the column permutation is the fill-reducing ordering and a separate row permutation records the pivot interchanges.

5.6 Static pivoting

The `ZeroPivotAction` policy governs near-singular pivots. `Fail` (the default) returns a rank-deficiency error on a pivot below the tolerance. `PerturbToEps` lifts any pivot of magnitude below a floor to that floor while preserving its phase, $d \mapsto d(\text{floor}/|d|)$, so the factorization satisfies $\mathbf{LDL}^\top = A + \Delta$ with a bounded Δ and never fails; the count of perturbed pivots is tracked per front. This never-fail factor is intended as a preconditioner for iterative refinement; the Krylov path that consumes it is outside the scope of this report.

5.7 Kernel scheduling and scoped pools

Four flop-count thresholds route the dense work, passed as a cheap `Copy KernelTuning` value rather than through global state: `scalar_gate` (≈ 4096 flops) below which an update runs as a scalar triple loop instead of a SIMD GEMM; `par_gemm` ($\approx 1 \times 10^6$) above which a `cmod` GEMM runs rayon-parallel; `par_cdiv` ($\approx 8 \times 10^6$) above which the panel-trailing and front GEMMs run parallel, which engages node-level parallelism on the large near-root fronts; and `use_gemm_schur`, which forces the scalar path for benchmarking. The tree recursion runs in a scoped rayon pool of the requested width rather than the global pool, so concurrent solves do not oversubscribe each other. The worker stack is sized to the assembly-tree depth, computed iteratively, `stack = clamp(32 KB · depth, 16 MB, 8 GB)`, which lets deep banded chain-trees factor without a stack overflow.

6 A-priori predictors

The analyze phase computes four predictors as exact functions of the symbolic structure and the scalar size $v = \text{value.bytes}$. They are the basis of the memory guarantee, the tuner, the thread policy, and the resource planner.

6.1 Per-path peak memory

The factor size is `factor_nnz · (v + 8)`, where the +8 is the CSC row index and the count is a structural upper bound because numeric cancellation can only lower it. The transient peak is bounded separately for the two paths.

For the left-looking path, a reference-count simulation of the panel-freeing schedule gives the live-panel peak `panel_live_peak.bytes`: it walks the postorder, adds each panel as it is assembled, and frees a panel when its refcount reaches zero, recording the running maximum of live panels plus already-compacted factor. This is the floor the solver actually operates at. A conservative whole-run bound assumes the parallel frontier holds all panels at once,

$$\text{scratch} = \frac{1}{4}(\text{panels_all} + \text{factor}) + 32 \text{ MB}, \quad (2)$$

$$\text{transient} = \text{panels_all} + \text{factor} + \text{input} + \text{scratch}, \quad (3)$$

where the scratch term covers the per-thread `cmod/cdiv` and emit buffers. For $\mathbf{LDL}^\top$ the panel size is `nrow · ncol · v`, the compacted factor is $(\frac{nc(nc+1)}{2} + \text{cnrow} \cdot nc)(v + 8)$ per supernode, and the input is `nnz(v + 8)`; for LU both L and U panels are held and the input is $2 \text{ nnz}(v + 8)$.

For the multifrontal path a level-parallel model takes, at each assembly-tree level, the sum of the front sizes $\sum \text{nrow}^2 v$ plus the contribution blocks of that level's children $\sum \text{cnrow}^2 v$, and multiplies the peak by $\frac{7}{5}$ to cover the work-stealing overlap of deep leaves of one subtree with mid-level fronts of another. Over the corpus both bounds hold with an estimate/measured ratio of about 1.5 in geomean and no under-prediction (Fig. 7), which is the property the memory guarantee requires; the panel-freeing floor is the tighter quantity used inside the tuner's veto.

6.2 Work and runtime

The geometric work factor $\text{flops} = \sum_s \text{nrow}_s^2 \text{ncol}_s$ is a type-independent proxy, so a single `f64` calibration serves all scalar types. The runtime estimate is $\text{factor_flops}/(\text{gflops} \cdot \text{speedup})$. The `Calibration` measures the single-thread geometric-flop rate and the parallel speedup once, by factoring a 24^3 seven-point Laplacian at one thread and at all physical cores, and caches the result keyed by a hardware fingerprint (core counts and a RAM-size bucket) in the manner of FFTW wisdom [14]. The speedup is interpolated linearly from one thread to the calibrated thread count and held flat beyond it, matching the saturating scaling of sparse-direct work; a fallback of 2 Gflop/s and $\sqrt{\text{cores}}$ speedup is used if the measurement fails.

6.3 Thread count

The thread-count predictor maps three structural quantities, the factor flops, the largest front height, and the peak tree width, to a worker count. A thin and narrow problem ($\text{front_nrow_max} < 512$ and $\text{tree_width_max} < 128$) is capped at two workers, where more only regress; a small problem ($\text{factor_flops} < 3 \times 10^8$) is capped at four, where scheduling overhead dominates; otherwise all cores are used. On the corpus this lands within about 10% of the per-matrix optimal in geomean, against about 50% for a fixed budget of two workers.

6.4 Structural features

The tuner consumes a structural fingerprint computed in the analyze phase. The pattern features, computed in $\mathcal{O}(\text{nnz})$, are n , nnz , the mean and maximum degree, the degree coefficient of variation (an arrow-pattern signal), the maximum and relative-mean bandwidth, and the diagonal-dominance and diagonal-presence fractions. The symbolic features, computed after analysis, are the supernode count, the fill nnz and fill ratio, the mean supernode width, the maximum front height, the tree depth, the maximum and mean tree width, the factor flops, the arithmetic intensity (flops per stored factor entry), and the fraction of flops in the largest and in the top 1% of fronts. These are the quantities that determine which configuration is fastest and smallest, and are the model’s input.

7 Tunable stages

Every analyze, factor, and kernel parameter is exposed through one flat `SolverSettings` struct, so a caller and the tuner address the same interface. Table 2 lists the parameters and the stage each affects. The analyze-stage parameters change fill and front shape; the factor-stage parameters change the transient schedule and the pivoting; the kernel-stage parameters change the scalar/SIMD and serial/parallel boundaries without changing the result. The tuner (Sec. 9) searches the ordering, `nemin`, relaxation width, `panel_nb`, the kernel thresholds, and the method; the remaining parameters (static pivoting, incomplete-factor drop tolerance, memory mode) are set directly by the caller.

8 Determinism

Three determinism properties hold. First, the numeric factorization is bit-identical regardless of the worker count: the parallel schedule changes the order in which independent subtrees and GEMMs execute, but not the accumulation within a pivot block, so a factor computed at one thread and at all cores agree to the last bit. The thread count is therefore a performance parameter only. Second, the analyze-phase choices (the auto ordering, the auto thread count, the amalgamation strategy) are deterministic functions of the matrix pattern. Third, the resource planner (Sec. 10) is a pure function of the memory estimate, the budget, and the cached calibration, so a plan is reproducible

Parameter	Stage	Effect
<code>ordering</code>	analyze	Fill-reducing ordering (Sec. 3)
<code>nemin</code>	analyze	Supernode amalgamation threshold
<code>relax</code>	analyze	Relaxed amalgamation width / extra-row budget
<code>reorder</code>	analyze	Child-reorder strategy (CB-stack peak vs. leaf parallelism)
<code>method</code>	factor	Left-looking or multifrontal
<code>threads</code>	factor	Worker count (fixed, or the auto predictor)
<code>on_zero_pivot</code>	factor	Exact fail or static-pivot floor
<code>drop_tol</code>	factor	Incomplete-factor threshold (preconditioner)
<code>memory</code>	factor	Factor emit / peak-RSS strategy
<code>panel_nb</code>	kernel	Bunch–Kaufman / LU panel width
<code>scalar_gate</code>	kernel	Scalar-loop vs. SIMD-GEMM flop gate
<code>par_gemm</code>	kernel	cmod GEMM parallel gate
<code>par_cdiv</code>	kernel	Front / trailing GEMM parallel gate
<code>use_gemm_schur</code>	kernel	SIMD vs. scalar Schur update

Table 2: The flat settings interface. Kernel-stage parameters change the schedule of the dense work but not the numeric result.

and can be computed before the matrix is factored. The pivot sequence is value-dependent (Bunch–Kaufman and threshold partial pivoting both branch on magnitudes), so re-factoring the same numeric matrix reproduces the same pivots; a sequence of matrices that share a pattern but differ in values may pivot differently unless the pivot order is fixed and reused, which is a planned option (Sec. 13). The calibration is a timing measurement and is not bit-reproducible, but once cached it makes subsequent planning on the same hardware reproducible.

9 Learned configuration tuner

The tuner selects a `SolverSettings` from the structural feature vector. A multilayer perceptron predicts (log factor_ms, log peak_mb) from the standardized features concatenated with an encoding of a candidate configuration (an ordering one-hot, `nemin`, relaxation width, `panel_nb`, `scalar_gate`, `par_cdiv`, and the method): a $32 \rightarrow 64 \rightarrow 32 \rightarrow 2$ network with ReLU hidden layers, trained offline on 3054 measured (matrix, configuration) samples with scikit-learn [15] and exported as JSON weights. Inference is pure Rust and runs at analyze time over a fixed candidate grid; the outputs are destandardized to milliseconds and megabytes. The selected configuration minimizes $w \log(\hat{t}) + (1 - w) \log(\hat{m})$, with the default weight $w = 0.7$; $w = 1$ and $w = 0$ give the speed and memory extremes.

The prediction is constrained by a deterministic guard stack, applied in order:

1. If factor_flops exceeds the training range (`flops.ood_cap` $\approx 1.42 \times 10^{10}$), the untuned default is used, so the model does not extrapolate.
2. A candidate whose predicted log_peak_mb exceeds the default’s by more than $\ln(1.02)$ is rejected.
3. A multifrontal candidate whose estimated transient exceeds the left-looking floor is rejected, using the deterministic estimate rather than the model.
4. The selected configuration is re-analyzed, and accepted only if its exact estimates satisfy $\text{fill} \leq 1.02 \times \text{default}$, $\text{flops} \leq 1.05 \times \text{default}$, and its memory floor stays under the default’s; otherwise the default is used.
5. The survivor must improve the score by at least 0.08, or 0.20 if it changes the factorization method.

The asymmetry between the memory and time guarantees is a consequence of what is predictable. Peak memory is a deterministic function of structure, so a candidate can be guaranteed to never exceed the default’s memory by the estimate-based checks (2)–(4). Factor time depends on the achieved BLAS-3 efficiency, which the model predicts only approximately, so time is optimized in aggregate and backed by the exact-flop proxy in check (4) but is not guaranteed per matrix. The same learned-prediction-with-a-deterministic-fallback structure appears in hardware branch prediction [16], where a misprediction costs only re-execution and not correctness.

10 Solver-in-the-loop and resource governance

Two features support running many solves under a fixed resource budget. First, the analyze and factor phases are separate: a symbolic object is computed once and factors any matrix that shares its pattern, so a sequence of same-pattern systems (a frequency sweep, a time-stepping or Newton iteration) pays the ordering and symbolic cost once and only re-runs the numeric factorization. Second, each factorization runs in a scoped thread pool of a caller-set width, so many concurrent solves coexist without oversubscribing the machine; the fixed-budget thread setting is the intended mode for concurrent solving, where the per-solve auto thread count would otherwise contend.

The resource planner `tuning::plan` turns a memory estimate, a budget (a memory ceiling and a thread budget), and the cached calibration into a plan: a predicted peak, a predicted runtime, and a decision on whether the factorization fits the ceiling. It is a pure function of its inputs. Because the memory estimate is an upper bound rather than a sample, a scheduler can pack concurrent solves against a memory ceiling without the safety margin an unbounded estimate would require. Each solve consumes a bounded memory and a thread width; the thread predictor supplies a scaling-aware width per solve, so non-scaling solves can run many-abreast at low thread counts while a BLAS-3-rich solve claims more cores. Because the per-solve peaks and runtimes are predicted, the aggregate memory footprint and the makespan of a batch are computable before the batch runs. The limits on this are the memory-bandwidth contention between concurrent factorizations, which the current calibration does not model (it measures per-solve speedup, not contention), the conservatism of the memory estimate, which reduces packing density, and NUMA placement on multi-socket machines.

11 Evaluation

The figures are produced by the `bench_suite` corpus engine and `superlu_corpus.py` over the SuiteSparse collection [24] (1k–100k DOFs; SPD, indefinite, unsymmetric, complex) on a 12-core / 24-thread machine. RSLAB, faer [22], MKL PARDISO [21], and SuperLU [23] were measured in one run, so the cross-solver numbers are not affected by run-to-run drift.

11.1 Scaling

Figures 1 and 2 plot factor time and peak memory against nnz with a least-squares power-law fit $\sim \text{nnz}^\alpha$ per solver (solves with residual above 0.1 excluded from the fit). The RSLAB curve is the auto-tuned default, which picks left-looking or multifrontal per matrix under the guards. Table 3 gives the fitted orders and the head-to-head geomean ratios on the matrices both solvers factor accurately.

RSLAB’s auto-tuned default is between PARDISO and faer on both axes: lower factor-time and memory growth than faer, higher than PARDISO’s sublinear orders. In the head-to-head it is about $7\times$ slower and $2.3\times$ larger than PARDISO, and about $7\times$ faster and $2.3\times$ smaller than faer.

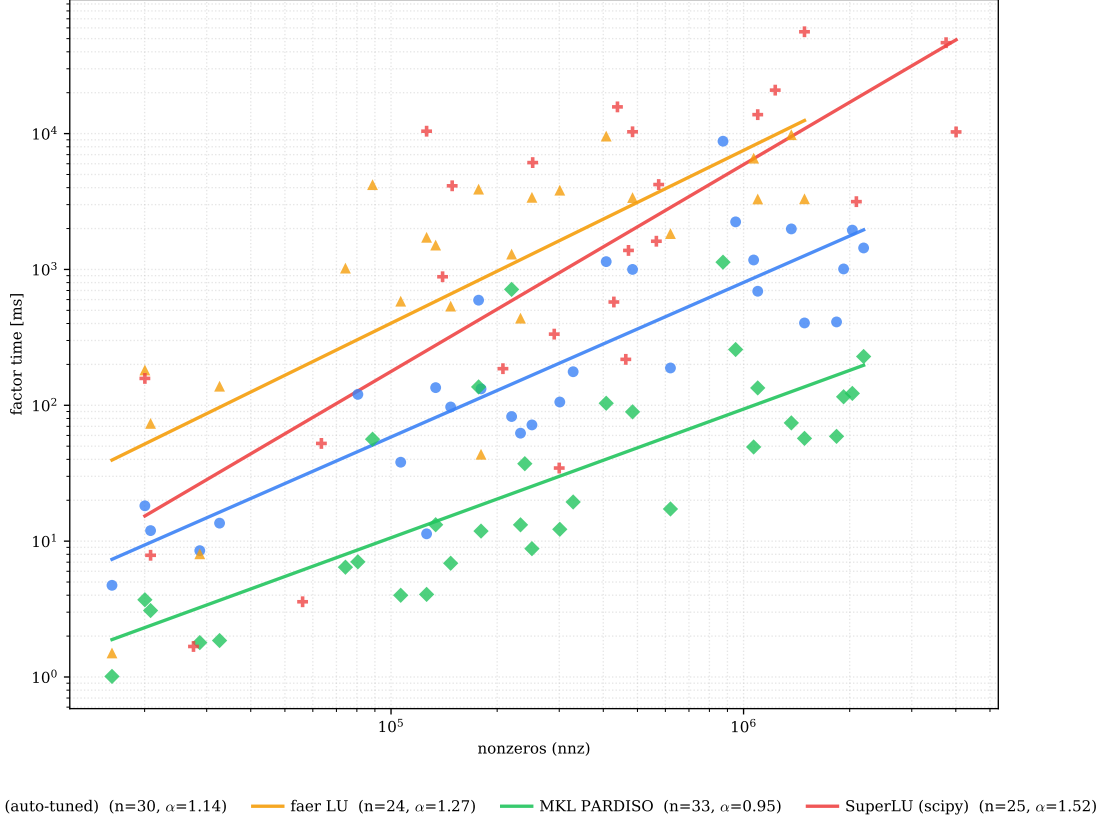


Figure 1: Corpus scaling of factor time vs. problem size, with power-law fits.

11.2 Accuracy

Figure 3 shows the relative residual $\|Ax - b\|/\|b\|$ per solver. Where RSLAB factors, 24 of 31 matrices are below 1×10^{-8} , matching PARDISO and ahead of faer. The exact $\mathbf{LDL}^\top$ declines a handful of indefinite saddle-point matrices rather than returning a degraded solution.

11.3 Configuration tuner

Figure 4 measures the tuned configuration against the untuned default across the corpus. The balanced tuner ($w = 0.7$) is $1.38\times$ faster at $0.81\times$ the memory, at zero of 89 memory regressions; the effect is larger on small matrices ($1.57\times$) than large ($1.22\times$). Figure 5 shows the three Pareto weights as a time–memory cloud relative to the default.

11.4 Thread scaling and estimator validation

Figure 6 shows per-solver thread scaling (mean over the corpus with a min–max band). Sparse-direct factorization is largely memory-bandwidth bound, so every solver saturates below linear, and the band is wide because some matrices regress past a few threads, which is why the worker count is predicted per matrix. Figure 7 validates the memory estimator against the measured peak of both paths, and Figure 8 shows the analyze/factor/solve split.

12 Related work

Automatic and learned tuning of sparse solvers has been studied along several axes. GPTune [17] tunes HPC libraries including SuperLU_DIST by Bayesian optimization with multitask learning across problem sizes, using online trial runs per problem; the RSLAB model is a feed-forward

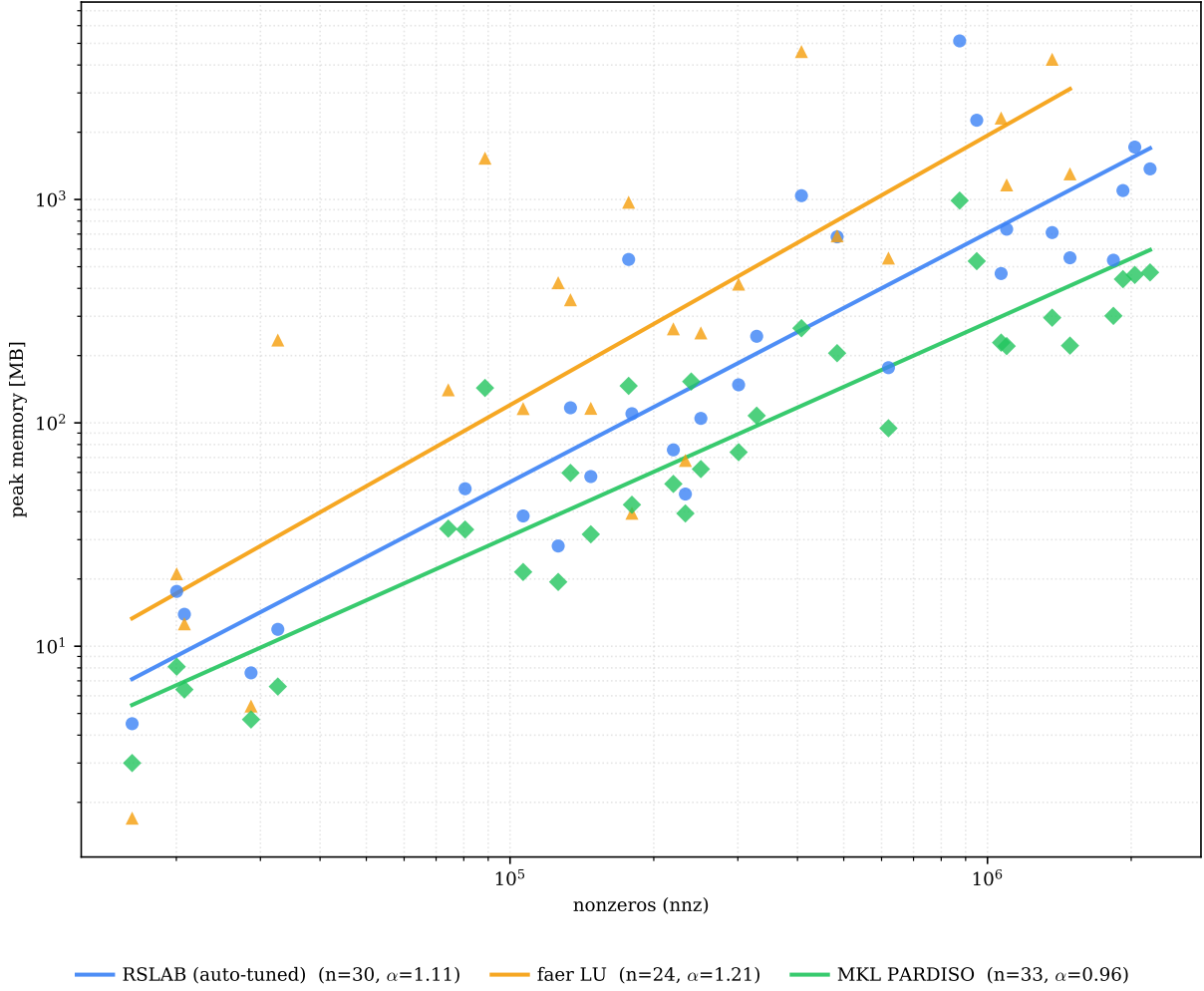


Figure 2: Corpus scaling of peak factor memory vs. problem size, with power-law fits.

regressor evaluated once at analyze time. OSKI [18] autotunes at the sparse-kernel level (register and cache blocking for sparse matrix-vector products), not at the solver-configuration level. Lighthouse [19] and the work of Bhowmick, Raghavan et al. [20] apply machine learning to select a solver or preconditioner from a portfolio, a classification problem, rather than to predict and tune a direct factorization’s configuration. Production direct solvers such as MKL PARDISO [21] and MUMPS ship hand-written heuristic automatic modes rather than learned models. The structure used here, a learned performance regressor constrained by a deterministic guarantee and an out-of-range fallback, is closest in form to hardware branch prediction [16], where a learned predictor is paired with a mechanism that makes a misprediction cost re-execution and not correctness.

13 Limitations and future work

The tuner’s gains are aggregate: the geomean speedup is $1.38\times$, and part of it is within the per-matrix single-shot timing variance of about $\pm 20\%$, so the geomean rather than any single matrix is the signal. The model is trained on one machine and one corpus and does not transfer across hardware without retraining, in the same way a branch predictor is tuned per microarchitecture; a meta-tuner that runs the sweep, the training, and the guard-threshold calibration on a user’s own matrix set to emit a problem-class profile is planned. Time is optimized but not guaranteed; only memory is. Planned tunable stages that keep the exact direct path are an RCM/band ordering [25],

solver	scaling order α		head-to-head vs. RSLAB	
	factor time	peak memory	factor time	peak memory
MKL PARDISO	0.95	0.96	$7.1\times$ slower	$2.3\times$ more
RSLAB (auto-tuned)	1.14	1.11	–	–
faer LU	1.27	1.21	$7.0\times$ faster	$2.3\times$ less
SuperLU (SciPy) [†]	1.52	–	$8.9\times$ faster	–

Table 3: Empirical scaling orders and head-to-head geomean ratios (matrices both solvers factor to below 0.1 residual). [†]SuperLU is an open-source reference on the time axis only: its peak memory is sampled process RSS from a SciPy child (`spilu` exposes no peak), which is not comparable to the in-process peaks and records 0 MB on fast factorizations, so it is omitted from the memory comparison; it runs single-threaded with a 60s per-matrix cap, so its 8 hardest corpus matrices are absent (a survivor bias that flatters its time), and it scatters over three orders of magnitude.

an exposed threshold-pivot tolerance, a tunable equilibration strategy [26], static pivot-order reuse for fixed-pattern sequences, parallel and blocked triangular solves, and two-dimensional front parallelism. Each adds a candidate dimension to the tuner, under the constraint that every stage provides an a-priori memory estimate so the deterministic memory guarantee continues to hold.

14 Conclusion

RSLAB is a pure-Rust, scalar-generic sparse direct solver that reaches within a single-digit factor of MKL PARDISO on the SuiteSparse corpus and is faster and smaller than the open-source alternatives. Its analyze phase computes a per-path memory bound, a work and runtime estimate, and a thread recommendation, all as exact functions of structure; a learned tuner selects the solver configuration from that structure under a deterministic guard stack that keeps its memory below the untuned default. The numeric result is independent of the worker count and the resource planner is a pure function of its inputs, which makes batched, solver-in-the-loop execution reproducible and its cost computable in advance.

References

- [1] P. R. Amestoy, T. A. Davis, I. S. Duff. An approximate minimum degree ordering algorithm. *SIAM J. Matrix Anal. Appl.* 17(4):886–905, 1996; and Algorithm 837: AMD. *ACM Trans. Math. Softw.* 30(3):381–388, 2004.
- [2] P. R. Amestoy. Recent progress in parallel multifrontal solvers and the approximate minimum fill (HAMF) ordering. Technical report / CERFACS, 1999.
- [3] E. Rothberg, S. C. Eisenstat. Node selection strategies for bottom-up sparse matrix ordering. *SIAM J. Matrix Anal. Appl.* 19(3):682–695, 1998.
- [4] A. George. Nested dissection of a regular finite element mesh. *SIAM J. Numer. Anal.* 10(2):345–363, 1973.
- [5] G. Karypis, V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM J. Sci. Comput.* 20(1):359–392, 1998.
- [6] C. M. Fiduccia, R. M. Mattheyses. A linear-time heuristic for improving network partitions. *19th Design Automation Conference*, 175–181, 1982.
- [7] J. D. Hogg, E. Ovtchinnikov, J. A. Scott. A sparse symmetric indefinite direct solver for GPU architectures (SSIDS). *ACM Trans. Math. Softw.* 42(1):1–25, 2016.
- [8] C. Ashcraft, R. Grimes. The influence of relaxed supernode partitions on the multifrontal method. *ACM Trans. Math. Softw.* 15(4):291–309, 1989.

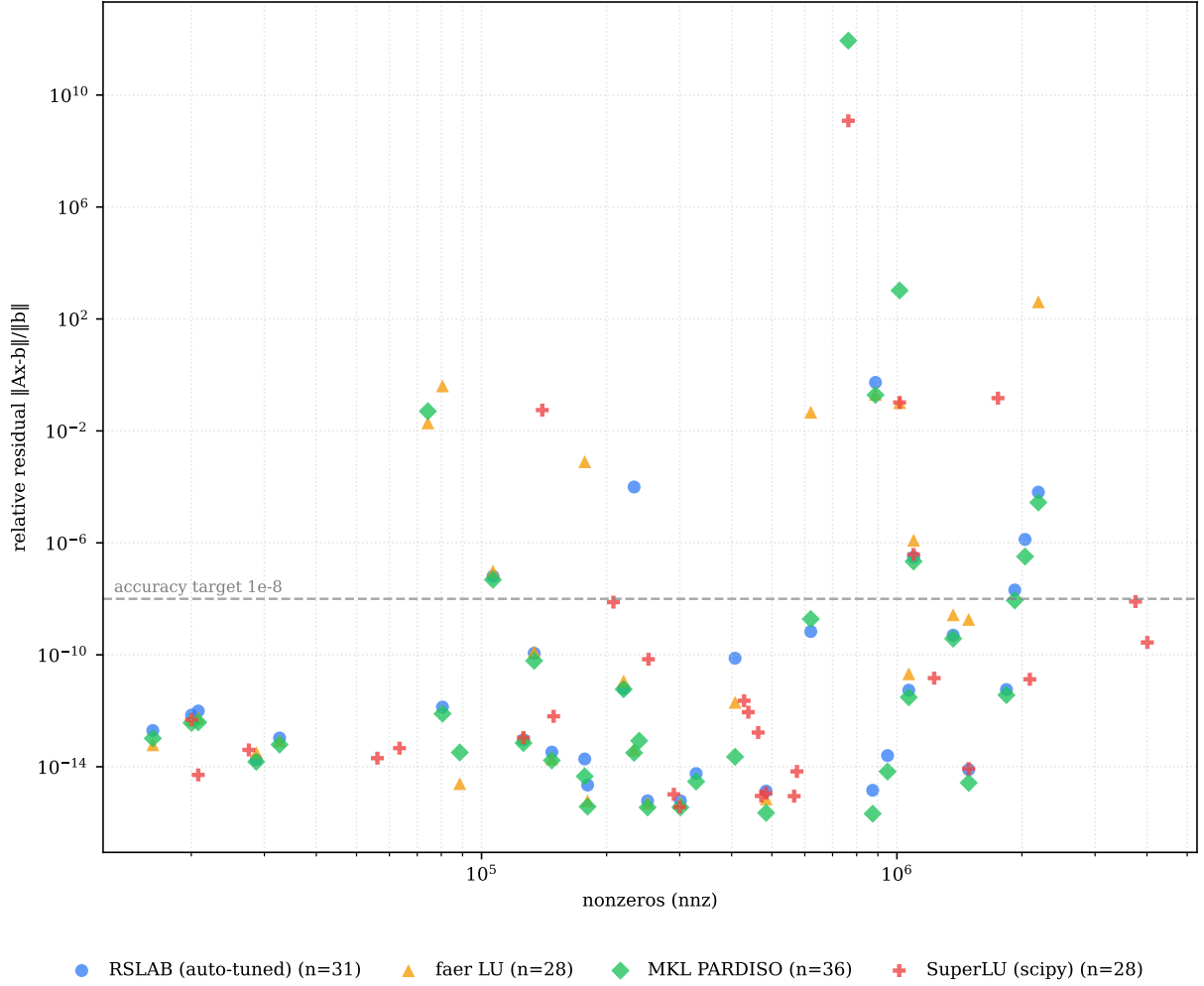


Figure 3: Relative residual across the corpus, per solver.

- [9] J. R. Bunch, L. Kaufman. Some stable methods for calculating inertia and solving symmetric linear systems. *Math. Comp.* 31(137):163–179, 1977.
- [10] I. S. Duff, J. K. Reid. The multifrontal solution of indefinite sparse symmetric linear equations. *ACM Trans. Math. Softw.* 9(3):302–325, 1983.
- [11] J. W. H. Liu. The multifrontal method for sparse matrix solution: theory and practice. *SIAM Review* 34(1):82–109, 1992.
- [12] J. W. H. Liu. On the storage requirement in the out-of-core multifrontal method for sparse factorization. *ACM Trans. Math. Softw.* 12(3):249–264, 1986.
- [13] T. A. Davis. Algorithm 832: UMFPACK, an unsymmetric-pattern multifrontal method. *ACM Trans. Math. Softw.* 30(2):196–199, 2004.
- [14] M. Frigo, S. G. Johnson. The design and implementation of FFTW3. *Proc. IEEE* 93(2):216–231, 2005.
- [15] F. Pedregosa et al. Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* 12:2825–2830, 2011.
- [16] D. A. Jiménez, C. Lin. Dynamic branch prediction with perceptrons. *7th Int. Symp. High-Performance Computer Architecture (HPCA)*, 197–206, 2001.
- [17] Y. Liu, W. M. Sid-Lakhdar, O. Marques, X. Zhu, C. Meng, J. W. Demmel, X. S. Li. GPTune: multitask learning for autotuning exascale applications. *PPoPP*, 234–246, 2021.

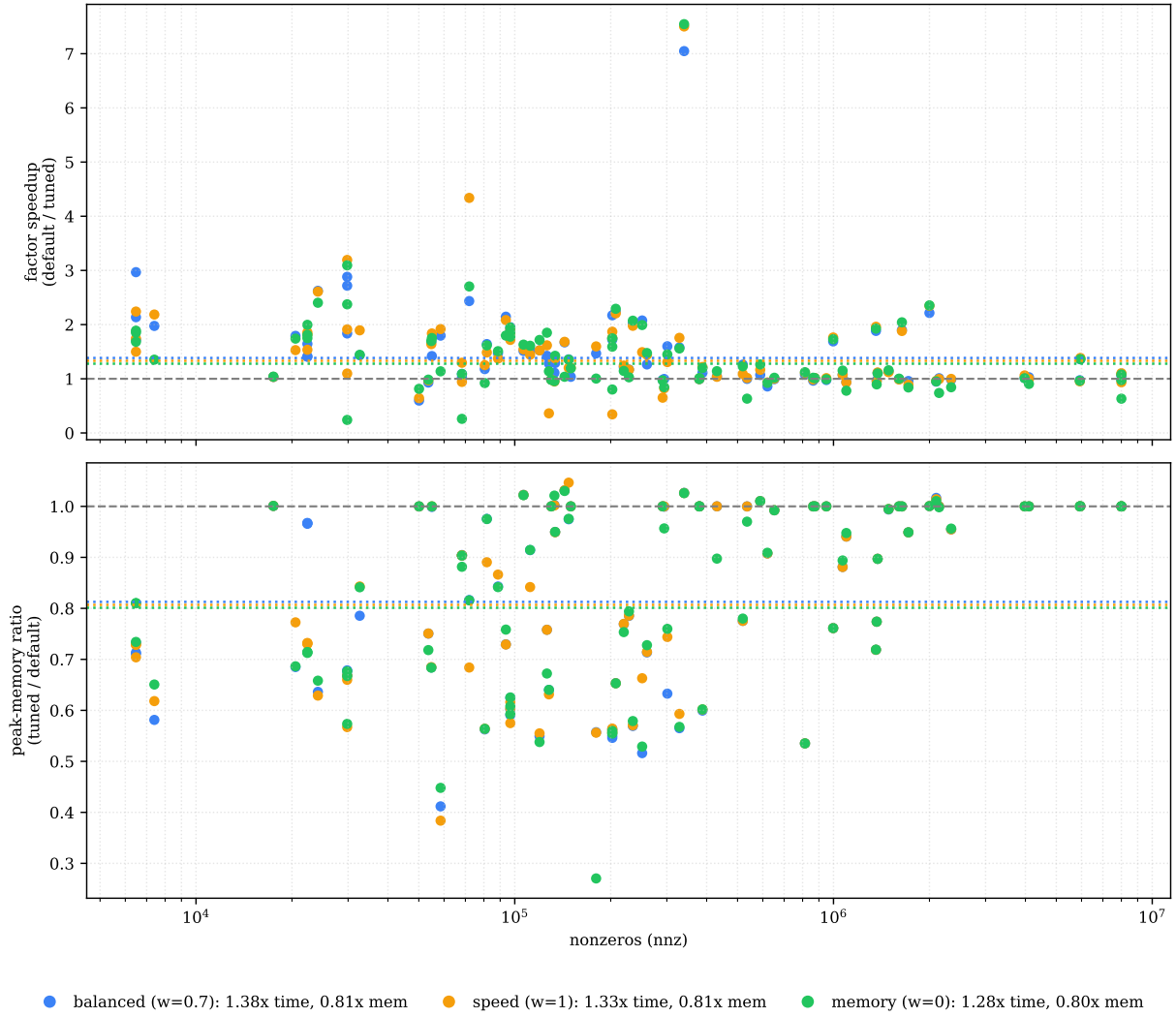


Figure 4: Factor speedup and peak-memory ratio of the tuned vs. untuned solver by problem size, three Pareto weights (dotted = geomean).

- [18] R. Vuduc, J. W. Demmel, K. A. Yelick. OSKI: a library of automatically tuned sparse matrix kernels. *J. Phys.: Conf. Ser.* 16:521–530, 2005.
- [19] E. R. Jessup, P. Motter, B. Norris, K. Sood. Performance-based numerical solver selection in the Lighthouse framework. *SIAM J. Sci. Comput.* 38(5):S750–S771, 2016.
- [20] S. Bhowmick, V. Eijkhout, Y. Freund, E. Fuentes, D. Keyes. Application of machine learning in selecting sparse linear solvers. *Int. J. High Perf. Comput. Appl.*, 2006.
- [21] O. Schenk, K. Gärtner. Solving unsymmetric sparse systems of linear equations with PARDISO. *Future Gener. Comput. Syst.* 20(3):475–487, 2004.
- [22] S. Quinones. faer-rs: a linear algebra library for the Rust programming language. <https://github.com/sarah-quinones/faer-rs>, 2024.
- [23] X. S. Li. An overview of SuperLU: algorithms, implementation, and user interface. *ACM Trans. Math. Softw.* 31(3):302–325, 2005.
- [24] T. A. Davis, Y. Hu. The University of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38(1):1–25, 2011.
- [25] E. Cuthill, J. McKee. Reducing the bandwidth of sparse symmetric matrices. *Proc. 24th ACM National Conference*, 157–172, 1969.

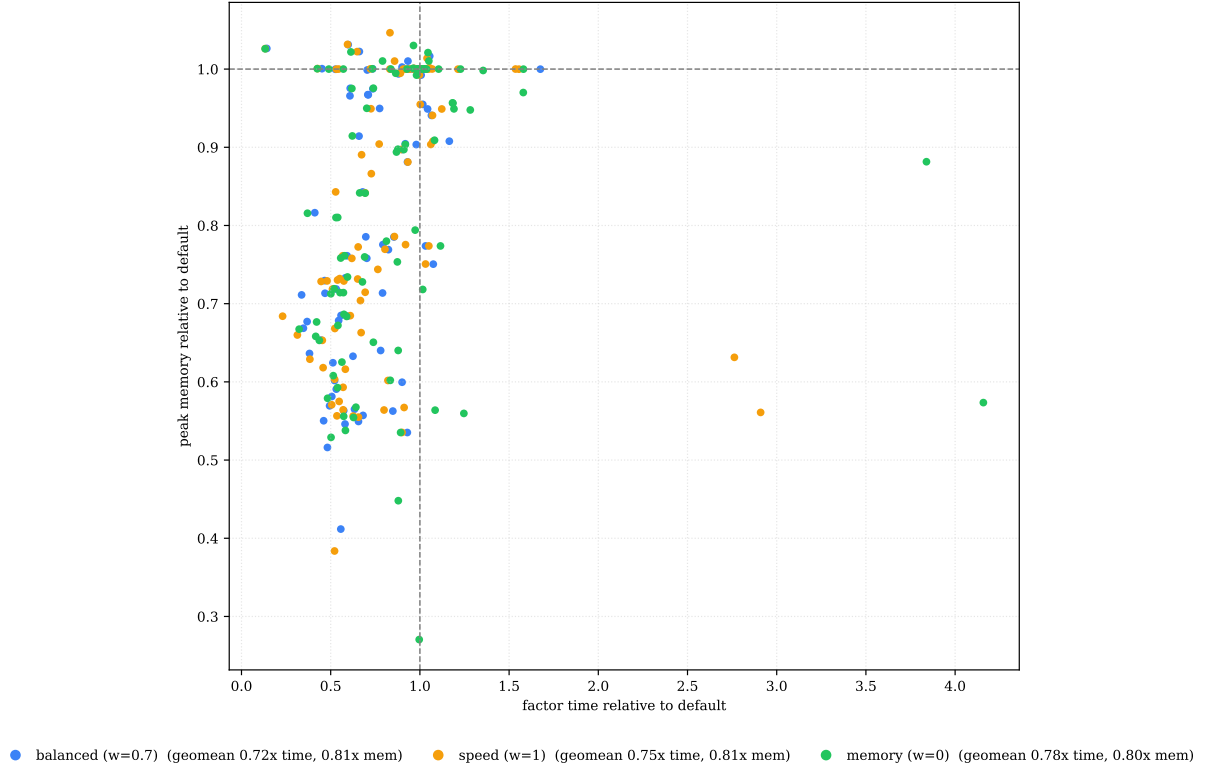


Figure 5: The tuner’s three Pareto modes as a time–memory cloud relative to the untuned default.

- [26] D. Ruiz. A scaling algorithm to equilibrate both rows and columns norms in matrices. Technical Report RAL-TR-2001-034, Rutherford Appleton Laboratory, 2001.

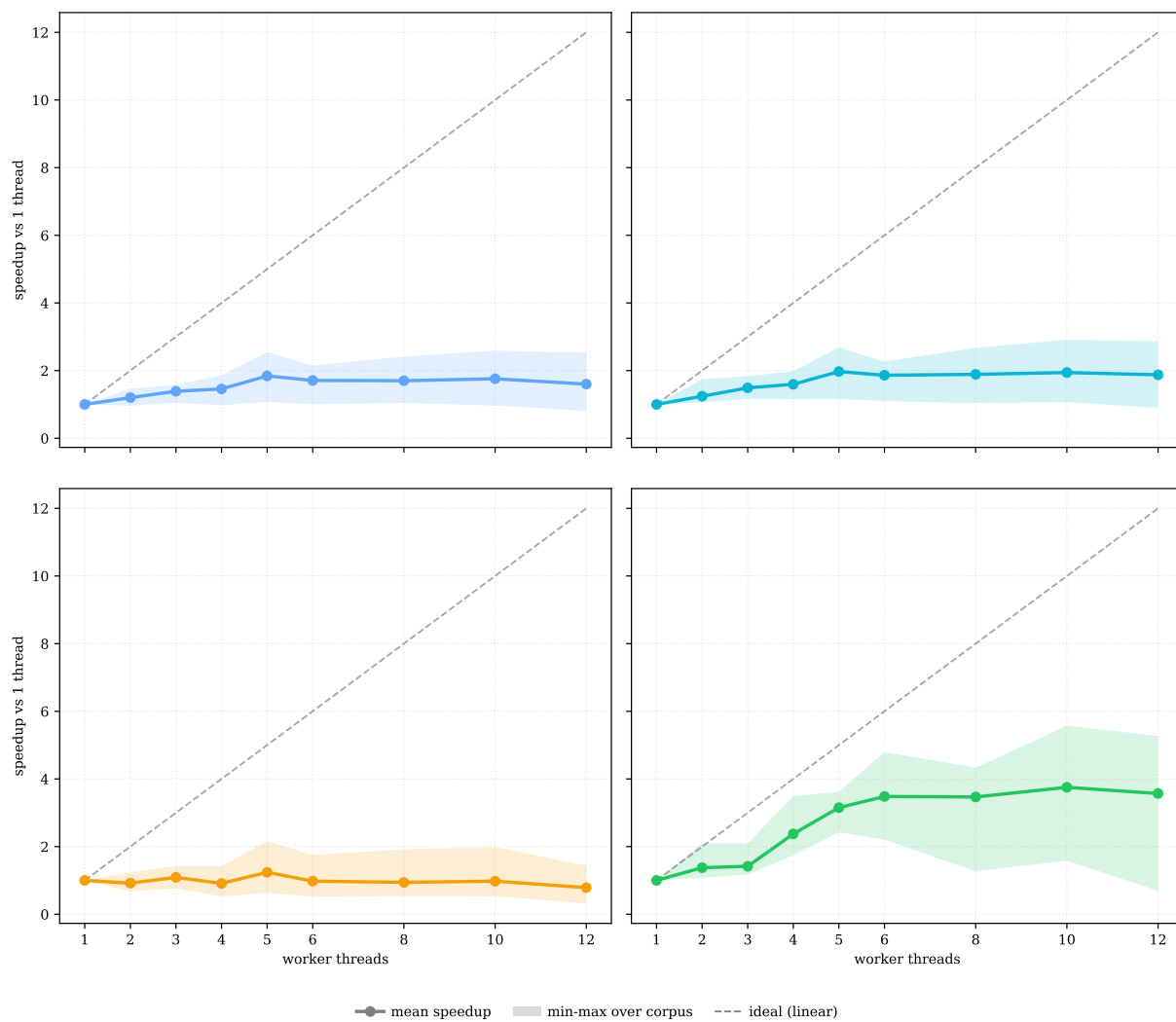


Figure 6: Thread scaling per solver, 1–12 workers, mean with min–max band over the corpus.

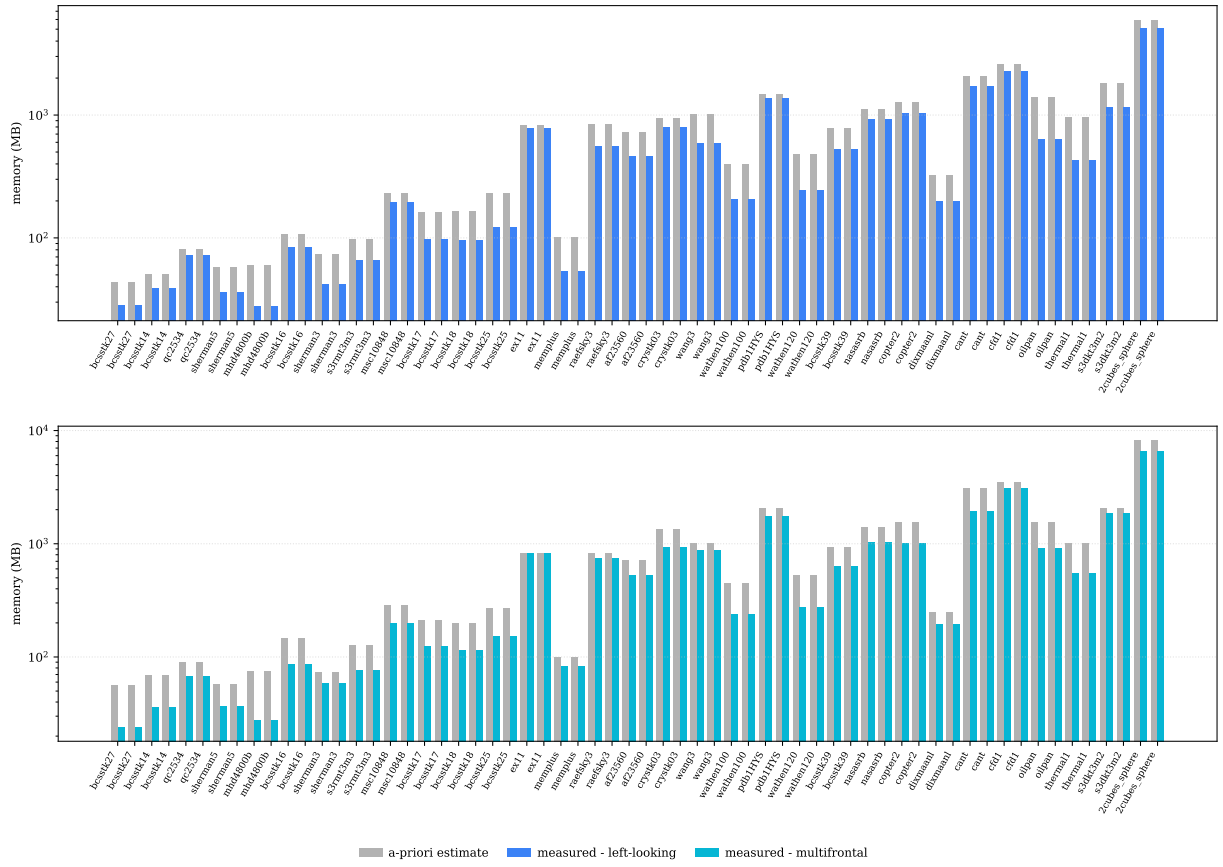


Figure 7: A-priori memory estimate (conservative bound and panel-freeing floor) vs. the measured peak, per RSLAB path. The estimate does not under-predict.

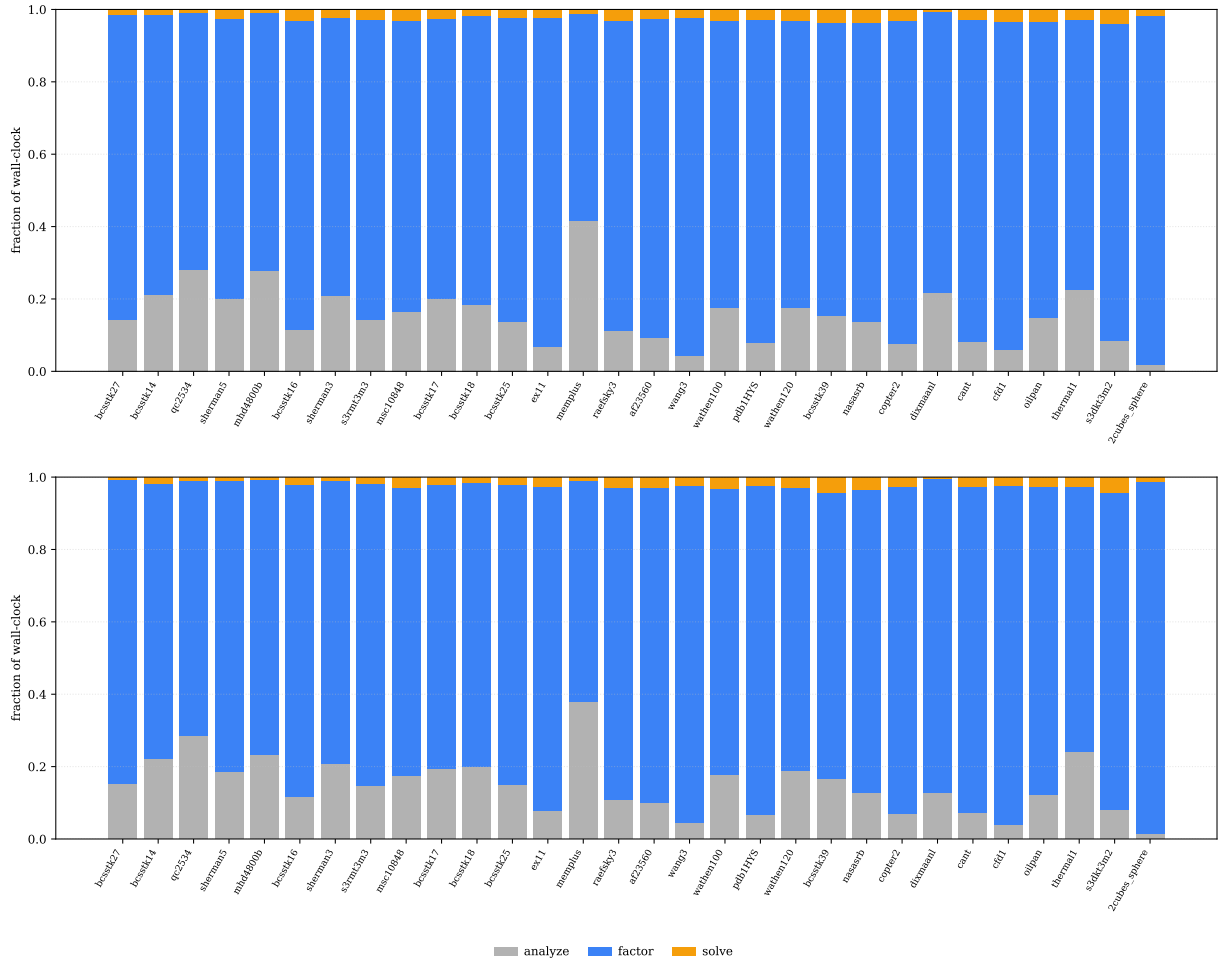


Figure 8: Analyze / factor / solve wall-clock split per matrix, per RSLAB path.